Reviewer Pack — Legendre-Symbol Partial-Sum Anomaly Excludes ACC^0

Entry bff6969c1719 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 08:01:01 UTC

Legendre-Symbol Partial-Sum Anomaly Excludes ACC^0

Entry ID: `bff6969c1719` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 08:01:01 UTC

1. Conjecture

Field A (mathematical branch): Analytic number theory: Polya-Vinogradov / Burgess partial sums of the Legendre symbol over Z_p **Field B** (complexity object): ACC^0[m] circuit complexity for explicit functions, framed via communication-matrix discrepancy and the Razborov-Rudich non-largeness escape

Statement:

> For f:{0,1}^n -> {-1,+1} index inputs as integers k in {0,...,2^n-1}, let p_n be the largest prime below 2^n, and let chi_n be the Legendre symbol mod p_n. Define Q(f) = (1/sqrt(p_n)) * max_{1 <= M < p_n} | sum_{k=1}^M f(bin_n(k)) * chi_n(k) |. Conjecture: (i) for uniform random f, Pr[Q(f) >= n^{1/4}] = o(1) (non-large, dodging Razborov-Rudich); (ii) every f computable by an ACC^0[m] circuit of size at most $2^{\{n}\{1/3\}}$ satisfies Q(f) <= (log n)^{O(1)}; (iii) the depth-3 Sipser function f_S^{(3)} with alternating OR/AND/OR of bottom fan-in ceil(n^{1/3}) satisfies Q(f_S^{(3)}) >= c * n^{1/8} for an absolute c>0. Therefore R(f) := [Q(f) >= n^{1/4}] is a non-large, polynomial-time-on-truth-table natural property excluding poly-size ACC^0[m].

Rationale (proposer's reasoning):

> ACC^0[m] functions are Z_2^n-Fourier-structured (Razborov-Smolensky, Bonami-Beckner) and possess approximate low-degree symmetric polynomial representations, which align with the additive group of indices but carry no multiplicative-character (Z_{p_n}^*) structure. Polya-Vinogradov gives sqrt(p) log p partial-sum cancellation for chi alone; twisting by an ACC^0 truth-table should preserve square-root cancellation since the two algebraic regimes (Z_2^n vs Z_p) interact pseudorandomly. Sipser's deeply-alternating gates create long monochromatic runs in the integer-indexed truth-table that resonate with the slow sign-change pattern of chi partial sums, breaking cancellation. Because random f also achieves cancellation (LIL gives Q ~ sqrt(log p_n)), the property [Q >= n^{1/4}] is satisfied by a vanishing fraction of random functions, sidestepping the natural-proofs largeness clause.

Taxonomy category: `COMM_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `17dd48d3e05d8a2a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For n in {10,12,14,16} with 30 seeds per family, the conjectured separation is SUPPORTED iff in >=3 of 4 n-values: median Q(C) >= 1.5 * median Q(B) AND max Q(A) < n^{1/4}. FALSIFIED if any n has Pr[Q(A) >= n^{1/4}] >= 0.2/30, or any (seed_B,seed_C) gives Q(B) >= Q(C).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.86	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.60	SAFE	SAFE
ALGEBRIZATION	SAFE	0.74	SAFE	SAFE
KARP_LIPTON	SAFE	0.85	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- Legendre symbol character sum natural proofs ACC0 lower bound - Polya-Vinogradov Burgess character sum circuit complexity discrepancy - Razborov-Rudich natural proofs ACC0[m] Sipser function character sum`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def sieve_of_eratosthenes(limit):
    primes = [True] * (limit + 1)
    p = 2
    while (p * p <= limit):
        if (primes[p] == True):
            for i in range(p * p, limit + 1, p):
                primes[i] = False
        p += 1
    prime_numbers = []
    for p in range(2, limit):
        if primes[p]:
            prime_numbers.append(p)
    return prime_numbers

def legendre_symbol(a, p):
    if a == 0:
        return 0
```

```
    elif a < 0:
        return -legendre_symbol(-a, p)
    elif a % 2 == 0:
        return legendre_symbol(2, p) * legendre_symbol(a // 2, p)
    else:
        if (p % 8 == 1 or p % 8 == 7):
            return pow(a, (p - 1) // 2, p)
        elif (p % 8 == 3 or p % 8 == 5):
            return -pow(a, (p - 1) // 2, p)

def fast_power(base, exp, mod):
    result = 1
    base = base % mod
    while exp > 0:
        if (exp % 2) == 1:  # If exp is odd, multiply base with result
            result = (result * base) % mod
        exp = exp >> 1  # Divide the exponent by 2
        base = (base * base) % mod  # Square the base
    return result

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [10, 12, 14, 16]
    results = []

    for n in n_values:
        p_n = sieve_of_eratosthenes(2**n)[-1]
        chi_n = [legendre_symbol(k, p_n) for k in range(2**n)]
        T_f_A = [random.choice([-1, 1]) for _ in range(2**n)]
        T_f_B = [random.choice([-1, 1]) for _ in range(2**n)]
        T_f_C = [random.choice([-1, 1]) for _ in range(2**n)]

        def Q(f):
            max_sum = 0
            current_sum = 0
            for k in range(1, 2**n):
                current_sum += f[k] * chi_n[k]
                if abs(current_sum) > max_sum:
                    max_sum = abs(current_sum)
            return max_sum / math.sqrt(p_n)

        Q_A = Q(T_f_A)
        Q_B = Q(T_f_B)
        Q_C = Q(T_f_C)

        results.append({
            "n": n,
            "Q_A": Q_A,
            "Q_B": Q_B,
            "Q_C": Q_C
        })

    median_Q_A = sorted([r["Q_A"] for r in results])[len(results) // 2]
    median_Q_B = sorted([r["Q_B"] for r in results])[len(results) // 2]
    median_Q_C = sorted([r["Q_C"] for r in results])[len(results) // 2]
```

```
    max_Q_A = max(r["Q_A"] for r in results)
    max_Q_B = max(r["Q_B"] for r in results)
    max_Q_C = max(r["Q_C"] for r in results)

    conjecture_holds = median_Q_C >= 1.5 * median_Q_B and max_Q_A < n_values[0] ** (1/4)
    counterexample = ""

    if not conjecture_holds:
        if max_Q_B >= max_Q_C > max_Q_A - median_Q_A or any(Q(T_f_A) >= n**0.25 for n in n_values)
            counterexample = "median Q_C < 1.5 * median Q_B or max Q(A) >= n^{1/4}"

    return {
        "metric_name": "Q",
        "metric_value": (median_Q_A + median_Q_B + median_Q_C) / 3,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 60, 2))
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_

    Q_A_values = [r["Q_A"] for r in results]
    Q_B_values = [r["Q_B"] for r in results]
    Q_C_values = [r["Q_C"] for r in results]

    mean_Q_A = sum(Q_A_values) / len(Q_A_values)
    std_Q_A = math.sqrt(sum((x - mean_Q_A) ** 2 for x in Q_A_values) / len(Q_A_values))
    mean_Q_B = sum(Q_B_values) / len(Q_B_values)
    std_Q_B = math.sqrt(sum((x - mean_Q_B) ** 2 for x in Q_B_values) / len(Q_B_values))
    mean_Q_C = sum(Q_C_values) / len(Q_C_values)
    std_Q_C = math.sqrt(sum((x - mean_Q_C) ** 2 for x in Q_C_values) / len(Q_C_values))

    support_fraction_A = sum(1 for q in Q_A_values if q >= n_values[0] ** (1/4)) / len(Q_A_values)
    support_fraction_B = sum(1 for q in Q_B_values if q >= n_values[0] ** (1/4)) / len(Q_B_values)
    support_fraction_C = sum(1 for q in Q_C_values if q >= n_values[0] ** (1/4)) / len(Q_C_values)

    if all(support_fraction_A < 0.2 and support_fraction_B < 0.2 and support_fraction_C < 0.2):
        print(f"RESULT: SUPPORTED mean_Q_A={mean_Q_A} std_Q_A={std_Q_A} support_fraction_A={suppor
    elif any(support_fraction_A >= 0.2 or support_fraction_B >= 0.2 or support_fraction_C >= 0.2):
        print(f"RESULT: FALSIFIED counterexample=\"median Q_C < 1.5 * median Q_B or max Q(A) >= n^
    else:
        print("RESULT: INCONCLUSIVE support_conditions_not_met")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5579605f.py", line 116, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5579605f.py", line 62, in run_trial
    chi_n = [legendre_symbol(k, p_n) for k in range(2**n)]
             ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5579605f.py", line 38, in legendre_symb
    return legendre_symbol(2, p) * legendre_symbol(a // 2, p)
           ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5579605f.py", line 38, in legendre_symb
    return legendre_symbol(2, p) * legendre_symbol(a // 2, p)
           ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5579605f.py", line 38, in legendre_symb
    return legendre_symbol(2, p) * legendre_symbol(a // 2, p)
           ^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with a RecursionError in the legendre_symbol implementation before any data was produced, so neither the support nor falsification criteria could be evaluated. | next: Rewrite legendre_symbol iteratively (or use Euler's criterion via pow(a,(p-1)//2,p)) to avoid recursion depth issues, then rerun the pre-registered protocol for n in {10,12,14,16}.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	237380
2	preregistration	claude_max	opus	0	0	7277
3	novelty	claude_max	opus	0	0	4438
4	novelty	claude_max	opus	0	0	9369
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14929
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25345
7	judge	claude_max	opus	0	0	5436

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 304174 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/bff6969c1719.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bff6969c1719.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/bff6969c1719.tar.gz` (if generated)